



COMPUTER SCIENCE DEPARTMENT

CS0025 (SOFTWARE ENGINEERING 1)

ACTIVI

TY 4

[AI – ASSISTED]

Comparative Analysis of Software Development Methodologies

(Individual Assessment)

Name:	Dawn Andrei Pamesa
Professor:	Doc Beau Gray Habal
Date Performed:	October 1, 2026
Date Submitted:	October 1, 2026



Purpose:

This individual technical assessment requires students to **compare and analyze two software development methodologies/approaches** and determine their similarities, differences, strengths, limitations, and appropriate applications.

Students must select **ONE (1)** of the following comparison topics:

- **Option A: Agile vs. Waterfall**
- **Option B: Scrum vs. Extreme Programming (XP)**

The assessment emphasizes **analysis and application**, rather than simply providing definitions. Students should demonstrate their ability to evaluate methodologies and justify which approach would be more appropriate under different software development conditions.

Instructions

Part A. Selected Comparison

Selected Comparison:

- ☐ Agile vs. Waterfall
- ☒ **Scrum vs. Extreme Programming (XP)**

- Scrum vs. Extreme Programming (XP), two Agile methods that are often treated as the same thing since both work in short iterations, both welcome changing requirements, and both came out of the same movement that produced the Agile Manifesto (Beck et al., 2001).

I selected this comparison because the difference between the two is much less obvious than Agile versus Waterfall. Comparing two opposites mostly shows that they are opposites. Comparing two Agile methods forces a closer look at what each one actually controls: Scrum mainly controls how a team organizes, plans, and inspects its work, while XP mainly controls how the team writes, tests, and integrates its code. I also work in both ways in my own projects, where I plan work in short cycles and lean heavily on automated tests and small, frequent commits, so I wanted to know which of those habits come from Scrum and which come from XP.

Part B. Overview of the Two Approaches

Scrum:

- **Definition and purpose.** Scrum, a lightweight framework for developing products in complex environments where not everything can be known up front. It is built on empiricism, meaning decisions are based on what the team observes rather than on a detailed upfront plan, and its purpose is to deliver a usable piece of the product (an Increment) at the end of every short cycle so the team and stakeholders can inspect it and adapt (Schwaber & Sutherland, 2020).
- **Core principles or characteristics.** Scrum rests on three pillars, transparency,



inspection, and adaptation, and five values: commitment, focus, openness, respect, and courage. It defines three accountabilities: the Product Owner, who represents the stakeholders and orders the Product Backlog; the Scrum Master, who facilitates the process, coaches the team, and removes impediments; and the Developers (the Development Team in our module), who build the Increment. Its artifacts are the Product Backlog, the Sprint Backlog, and the Increment. However, Scrum deliberately does not say how to write or test code; it is a management framework, not an engineering method.

- **General development process.** Work is done in Sprints of one month or less, with two weeks being the most common length (CS0025 Module 3). Each Sprint starts with Sprint Planning, where the team sets a Sprint Goal and selects backlog items. A 15-minute Daily Scrum is held to check progress toward that goal and raise impediments. At the end, the Sprint Review shows the Increment to stakeholders and the Sprint Retrospective looks at how the team itself can improve. Backlog refinement continues throughout.
- **Typical project environment.** Product development with ten or fewer people per team, where requirements will emerge over time, stakeholders can attend regular reviews, and management wants visibility into progress. Since it does not depend on any programming practice, it is also used outside software.

Extreme Programming (XP):

- **Definition and purpose.** Extreme Programming, a software development discipline created by Kent Beck in the late 1990s. Its name comes from taking practices that are already known to be good and doing them to the extreme: if code review is good, review code all the time through pair programming; if testing is good, test all the time by writing tests first; if integration is good, integrate many times a day. Its purpose is to produce high quality software that can keep changing cheaply, at a pace the team can sustain (Beck & Andres, 2004).
- **Core principles or characteristics.** XP is based on five values: communication, simplicity, feedback, courage, and respect. These are put into practice through concrete technical practices, among them user stories, the weekly and quarterly cycles, pair programming, test-first programming, continuous integration, a ten-minute build, incremental design with refactoring, shared code ownership, and a whole team that includes a real customer (Beck & Andres, 2004). XP is very helpful when customer demands constantly change and it advocates frequent releases in short development cycles (CS0025 Module 3).
- **General development process.** Our module describes six phases: Planning, where user stories are captured (including a "parking lot" of stories) and prioritized; Analysis; Design; Execution, where coding and unit testing happen; Wrapping; and Closure, where the pilot launch, training, and production launch happen. Day to day, the customer chooses which stories go into a short cycle, programmers write a failing test first, write the code in pairs until it passes, refactor, and integrate the change into the main code base several times a day, so a small working release is always close.
- **Typical project environment.** Small, co-located teams of developers working with a customer who is available often, on projects where requirements are vague or change frequently, technical risk is high, and automated testing is



possible.

Part C. Comparative Analysis

Comparison Criterion	Scrum	Extreme Programming (XP)	Analysis
Development Approach	Iterative and incremental management framework based on empirical process control. Work is done in fixed-length Sprints; no engineering practices are prescribed.	Iterative engineering discipline built on short weekly cycles inside longer quarterly cycles. Prescribes specific coding and testing practices.	Scrum says how to organize and inspect the work but not how to build it; XP fills exactly that gap. This makes them complementary more than competing: Scrum alone can deliver poorly engineered Increments, and XP alone gives little structure for managing stakeholders.



FEU Institute of Technology

COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Comparison Criterion	Scrum	Extreme Programming (XP)	Analysis
Requirements	Kept as an ordered Product Backlog owned by one person, the Product Owner, and refined continuously.	Written as user stories together with the customer, who prioritizes them. Acceptance tests define when a story is done.	Both accept evolving requirements. Scrum funnels them through a single accountable person, which helps when many stakeholders compete. XP relies on direct customer and developer conversation, which is faster but only works if the customer is available and decisive.
Planning	Sprint Planning sets a Sprint Goal and selects items for the Sprint (about 2 hours for a 2-week Sprint in our module). A Product Goal guides the longer term.	The planning game: quarterly (release) planning plus weekly planning. Developers estimate, the customer chooses scope, and slack is built into each cycle.	Scrum's plan is anchored on a Sprint Goal, which gives the team focus and gives management something to track. XP plans lighter and more often, so the plan corrects itself faster but is less visible to people outside the team.



FEU Institute of Technology

COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Comparison Criterion	Scrum	Extreme Programming (XP)	Analysis
Team Structure	Three accountabilities: Product Owner, Scrum Master, and Developers. Self-managing and cross-functional, ten or fewer people.	Whole team: programmers, a real customer, testers, a coach, and often a tracker. No formal hierarchy; programmers pair and rotate, and everyone owns all the code.	Scrum names an explicit owner of the process (the Scrum Master), which helps new or less mature teams. XP depends more on experienced developers, but pairing and shared ownership spread knowledge so the project does not depend on one person.



Customer/Stakeholder Involvement	Stakeholders are represented by the Product Owner and formally inspect the product at every Sprint Review.	The customer is part of the team, writes stories and acceptance tests, and is available almost daily.	XP gives tighter feedback but demands much more of the customer's time. Scrum is more realistic when the customer cannot sit with the team, though a proxy Product Owner can filter or distort what users actually want.
Handling of Changes	New requests go to the Product Backlog. The Sprint Backlog can be renegotiated, but the Sprint Goal is protected; an obsolete Sprint can be cancelled by the Product Owner.	Changes are welcome every cycle. A story that has not been started can be swapped for one of equal size. Refactoring and automated tests keep the code cheap to change.	Scrum protects the team from churn in the middle of a Sprint, which gives stability and focus. XP reacts faster, but its responsiveness is technical: without its tests and refactoring, frequent change would quickly degrade the code. In Scrum, responsiveness is procedural.



FEU Institute of Technology

COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Documentation	Not prescribed. The artifacts and the Definition of Done must be transparent; teams add documents when their Definition of Done requires them.	Minimal. The code, unit tests, and acceptance tests serve as living documentation; story cards are often discarded once done.	Both follow the Agile value of working software over comprehensive documentation. Scrum can absorb formal documentation through the Definition of Done more easily, while XP can clash with regulated or audited settings unless tests and build logs are accepted as evidence.
Testing/Quality Assurance	Not prescribed. Quality is defined by the team's Definition of Done, which every Increment must meet.	Test-first programming, automated unit tests, customer acceptance tests, continuous integration, a ten-minute build, and pair programming as continuous code review.	This is the biggest practical difference. XP builds quality into every hour of work, while Scrum's quality is only as good as the Definition of Done the team writes. This is why many Scrum teams borrow XP's engineering practices (Kniberg, 2015).



FEU Institute of Technology

COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Delivery/Release	A usable Increment every Sprint of one month or less. The Product Owner decides when to release, and a release can happen more than once per Sprint.	Small, frequent releases. Code is integrated many times a day and can be deployed as often as daily.	XP shortens the feedback loop further. Scrum's fixed cadence is more predictable and easier for stakeholders and management to plan around.
Risk Management	Empirical inspect-and-adapt at every event. Short Sprints limit how much work is at risk; the Daily Scrum surfaces blockers and the Scrum Master removes them.	Spikes for unknown technology, test-first programming, continuous integration, simple design, and pair programming reduce technical risk; small releases and slack reduce schedule risk.	Scrum mainly addresses product and process risk (building the wrong thing, blocked teams). XP mainly addresses technical risk (defects, broken integrations, brittle design). A project exposed to both kinds benefits from combining them.
Advantages	Clear roles and accountability, a predictable cadence, high visibility for stakeholders, widely adopted, and usable outside software.	High code quality and fewer defects, very fast feedback, code that stays easy to change, knowledge sharing through pairing, and a sustainable pace.	Scrum's advantages are organizational and XP's are technical, so the right choice depends on where the project is weakest.



Limitations	Gives no engineering guidance, so technical debt can pile up behind on-time Sprints (Fowler, 2009). Depends heavily on a good Product Owner, and its events can turn into empty rituals.	Demands strong discipline and skill. Pair programming is often seen as costly, it needs an available customer, and it fits large, distributed, or documentation-heavy projects poorly.	Scrum's limitations come from what it leaves undefined; XP's come from how much it demands from the people doing it.
Suitable Project Types	Product development with evolving requirements, several stakeholders, a need for management visibility, and teams of ten or fewer (or several such teams).	Small co-located teams, rapidly changing requirements, technically risky or quality-critical code, and settings where automated testing is possible and the customer is reachable.	Suitability depends mostly on team size, how available the customer is, and whether the main risk is deciding what to build (favoring Scrum) or building it correctly (favoring XP).

Part D. Scenario-Based Application

Scenario 1 - Stable Requirements

A company needs a system with clearly defined requirements. The scope is unlikely to change significantly during development, and management requires detailed documentation and approval at major stages.

Which approach would you recommend? Why?

Recommendation: Scrum

Stable requirements, heavy documentation, and approval at major stages, a situation that honestly fits a plan-driven model like Waterfall best (Sommerville, 2016). However, between Scrum and XP, Scrum is clearly the better fit, for the following reasons:

- The clearly defined requirements can be written directly into an ordered Product Backlog that management reviews and approves before development starts, with the Product Owner accountable for it.
- Each Sprint Review works as a natural approval point. Major stages can be mapped to a set of Sprints, and management signs off on the Increment shown at the review before the next stage begins.
- The required documentation can be made part of the Definition of Done, so an item is not considered done until its design notes, user documentation, and sign-off exist.
- Since the scope is stable, Scrum's rule of protecting the Sprint Goal works in its favor: the team can plan Sprints with confidence and management gets a predictable cadence of progress reports.



XP is less suitable here. Its biggest strength, absorbing frequent change cheaply, is not needed when the scope barely moves, and its habit of treating code and tests as the documentation goes against a management that wants detailed written documents and formal gates. Yet XP's test-first programming and continuous integration could still be used inside the Scrum Sprints to raise quality, since Scrum leaves the engineering practices open.

Scenario 2 - Changing Requirements

A software startup is developing a new application. User requirements are expected to change frequently based on customer feedback, and the team needs to release working features regularly.

Which approach would you recommend? Why?

Recommendation: XP

A startup, small team, requirements that change often based on customer feedback, and a need to release working features regularly, which is the situation XP was originally designed for. The reasons are as follows:

- Weekly cycles and small releases mean customer feedback can reach the code within days, not at the end of a Sprint.
- The customer (or the founder acting as the customer) sits with the team, writes the user stories, and can swap any story that has not been started yet, so a change does not have to wait for the next planning event.
- Test-first programming, refactoring, and continuous integration keep the code safe to change over and over. Without them, a codebase that changes this frequently becomes fragile and slows the startup down, which is a common way young products fail.
- Pair programming and shared code ownership mean the small team is not blocked when one person is out, which matters when there are only a few developers.

Scrum is workable but less suitable. Because the Sprint Goal is protected, new feedback usually waits until the next Sprint, which can be up to a month away. Its formal accountabilities and events are also extra overhead for a team of three or four people, and Scrum by itself does nothing to keep the code changeable. However, if the startup grows and gains investors and several stakeholders, adding a Product Owner and Sprint Reviews on top of the XP practices would be a natural next step.

Scenario 3 - Your Own Scenario

Create one realistic software development scenario where one of your selected approaches would be more suitable than the other.

For students selecting Scrum vs. XP: Analyze the scenarios specifically according to Scrum and XP practices rather than treating them as general Agile vs. traditional development comparisons.

1. The project situation. Andy Trader, an automated cryptocurrency trading bot that I am building as a personal project. It runs in paper-trading mode first, where orders are only simulated, and live execution with real funds stays locked behind my manual approval. The team is small: I am both the developer and the only user, and I work with AI coding agents (Claude Code and Codex) that write and review code under my direction. Requirements change often as I learn from the paper-trading results, and a single bug in position sizing, leverage, or order logic could lose real money.



2. The selected approach. Extreme Programming (XP).

3. Why it is more appropriate. The dominant risk in this project is technical correctness, which is exactly what XP's practices target:

- Test-first programming for the order-sizing, risk-limit, and leverage logic, so every rule that protects money exists as an automated test before the code is written.
- Continuous integration and a fast build, so every change, including code written by an AI agent, is tested before it reaches the main branch.
- Pair programming has a direct equivalent in an agentic workflow: one agent writes the code while I, or a second agent, review it as the navigator. It keeps a second set of eyes on every change.
- Small releases match the paper-trading-first approach: features go out in small steps, are watched in simulation, and only then move toward live trading.
- The "real customer" practice is fully satisfied, since I am the user and I am always available to decide priorities.

4. Why the alternative is less suitable. With one human on the team, Scrum's three accountabilities collapse into the same person, so the Product Owner, Scrum Master, and Developer roles stop meaning anything, and events like the Daily Scrum become an empty ritual. More importantly, Scrum says nothing about testing, which is the main risk here. A two-week Sprint is also slower than the daily feedback I already get from paper trading. Scrum would add process without reducing the risk that actually matters.

Part E. Decision and Recommendation

No. Neither methodology is always better, since they were designed to solve different problems. Scrum is a framework for managing work under uncertainty; it gives a team roles, a cadence, and points of inspection, but leaves the engineering to the team. XP is a discipline for building software that can keep changing; it gives a team strict technical practices, but very little structure for managing stakeholders or reporting to management. A methodology is only "better" relative to the project's biggest risk: when the risk is building the wrong thing or losing visibility, Scrum is stronger, and when the risk is defects and a codebase that cannot keep up with change, XP is stronger. This is also why the two are commonly combined, with Scrum as the management layer and XP practices inside each Sprint (Kniberg, 2015).

Recommendation. Based on the analysis, my decision rule is to look at where the project's main risk is and choose accordingly. Listed below are the conditions for each factor:

Factor	Choose Scrum when...	Choose XP when...
Stability of requirements	Requirements are fairly stable within a Sprint, so protecting the Sprint Goal costs little.	Requirements change weekly or even daily from user feedback.
Project complexity	Complexity is mainly in the product and business (many features, many stakeholders).	Complexity is mainly technical (risky algorithms, integrations, correctness).
Customer involvement	The customer is busy or distant, so a Product Owner must act as their proxy.	A real customer can work with the team almost daily.



Factor	Choose Scrum when...	Choose XP when...
Team size and capabilities	Teams of up to ten, including newer teams that benefit from a Scrum Master guiding the process.	Small, co-located, technically disciplined teams comfortable with pairing and test-first programming.
Need for documentation	Management or clients expect written artifacts that can be required by the Definition of Done.	Tests and code are accepted as the main documentation.
Frequency of delivery	A predictable release every one to four weeks is enough.	Releases are needed in very small steps, as often as daily.
Quality requirements	Normal quality needs, enforced through a strong Definition of Done.	Defects are costly (money, safety, data), so quality must be built in through tests and continuous review.
Ability to respond to change	Change can wait for the next Sprint Planning.	Change must be absorbed immediately without degrading the code.

In short, I would choose Scrum for a team building a product for several stakeholders who need regular, visible checkpoints, and XP for a small team writing quality-critical code under constant change. When a project has both kinds of risk, which is common in practice, my recommendation is to run Scrum as the management framework and adopt XP's test-first programming, continuous integration, and refactoring as part of the team's Definition of Done.

Part F. Conclusion

- To conclude, the most important difference I found is that Scrum and XP work at different levels. Scrum manages the work: who decides priorities, how a Sprint is planned, and when the product is inspected. XP manages the code: how it is tested, reviewed, integrated, and kept easy to change. Most of the other differences in the matrix (planning, customer involvement, documentation, testing) follow from that one.

Scrum's strengths are clear accountability, a predictable cadence, and visibility for stakeholders, while its main limitation is that it leaves engineering quality entirely up to the team. XP's strengths are high code quality and the ability to change direction quickly, while its main limitation is how much discipline and customer time it demands, which makes it harder to scale.

What I learned is that selecting a methodology is not about choosing the most popular one, but about matching it to the project's biggest risk, its team, and its customer. Personally, this also explained my own workflow: the short planning cycles I use come from Scrum, and the tests and small commits I rely on come from XP. Yet the two work best together, which is why the best answer is often not Scrum or XP, but Scrum with XP.



Part G. References

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., et al. (2001). Manifesto for Agile Software Development. <https://agilemanifesto.org/>

Beck, K., & Andres, C. (2004). Extreme Programming Explained: Embrace Change (2nd ed.). Addison-Wesley.

FEU Institute of Technology. (2026). CS0025 Software Engineering 1, Module 3: Agile Methodologies (M3S0 to M3S2) [Course material].

Fowler, M. (2009). Flaccid Scrum. <https://martinfowler.com/bliki/FlaccidScrum.html>

Kniberg, H. (2015). Scrum and XP from the Trenches (2nd ed.). InfoQ. <https://www.infoq.com/minibooks/scrum-xp-from-the-trenches-2/>

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. <https://scrumguides.org/scrum-guide.html>

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

Part H. AI Interaction Log (if applicable)

AI Tool Used	Purpose	Sample/Key Prompt	Summary of AI Output	How You Verified/Revised the Output
Claude Code (Claude Opus 5.5), run as an agent in the Claude desktop app inside my personal Obsidian knowledge vault	Read the TA4 sheet and my own CS0025 Module 3 reviewer, and plan the paper	"Let's do Option B. You know the instructions. Use that PDF as the template, don't invent something new."	Extracted the nine required parts and the rubric from the sheet and pulled the Scrum and XP facts (roles, timeboxes, XP's six phases) from my Module 3 reviewer	I checked the extracted requirements against the sheet. When the agent started rebuilding the layout on its own, I stopped it and converted the official PDF to DOCX myself so the actual template is used.



AI Tool Used	Purpose	Sample/Key Prompt	Summary of AI Output	How You Verified/Revised the Output
Claude Code	Draft the overview and the comparison matrix	"Compare Scrum and XP on every criterion in the matrix. The Analysis column must explain why the difference matters, not just that they differ."	Drafted the definitions, the matrix entries, and the analysis column	Checked the Scrum content against the 2020 Scrum Guide (accountabilities, events, timeboxes, team size) and the XP values and practices against Beck and Andres (2004), and aligned the numbers with our module (two-week Sprints, 2-hour planning).
Claude Code	Apply the scenarios and form the recommendation	"Answer the scenarios using specific Scrum and XP practices. For Scenario 3, use my Andy Trader trading bot project."	Drafted Scrum for Scenario 1, XP for Scenarios 2 and 3, and the decision table for Part E	I chose the Scenario 3 project and its details from my own work, and checked that every answer argues from named practices (Sprint Goal, Definition of Done, test-first, continuous integration) rather than a generic Agile vs. Waterfall argument.
Claude Code using tools (Python with python-docx, Microsoft Word)	Fill in the official template and export the PDF with the required file name	"Fill in the DOCX template I converted."	A script inserted the answers under each part, filled the cover, matrix, and log tables, and exported the PDF	I reviewed the final PDF section by section and confirmed that each reference is a real source.

AI may assist you with activities such as:

- Brainstorming comparison criteria;
- Clarifying concepts;
- Organizing ideas;
- Generating possible scenarios for further analysis;
- Improving the clarity of your writing.



FEU Institute of Technology

COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

You are responsible for checking the accuracy of AI-generated information against your course materials and credible sources.

If no Generative AI tool was used, indicate: "Not Applicable – No generative AI tools were used."

Part I. AI Usage Declaration

Complete the following declaration:

AI Usage Declaration

I declare that any use of Generative AI in completing this assessment has been properly disclosed in my **AI Interaction Log**. I understand that AI serves only as an assistive tool and that I remain responsible for verifying the accuracy, relevance, and credibility of all information included in my submission.

I confirm that the final comparison, analysis, recommendations, and conclusions reflect my own understanding and judgment.

Student Name: Dawn Andrei Pamesa

Signature: Dawn Andrei Pamesa

Date: October 1, 2026